

L'intelligence artificielle au service des développeurs

Redha Moulla

16 - 18 septembre 2026

Plan de la formation

- Qu'est-ce que l'intelligence artificielle ?
- Éléments de machine learning et de deep learning
- Embeddings et LLM
- Les bonnes pratiques de prompting
- Les RAG et les Agents
- L'IAG dans le cycle de développement logiciel
- Enjeux éthique, de sécurité et de conformité de l'IAG

Qu'est-ce que l'intelligence artificielle ?

Définition littérale de l'intelligence artificielle

1. Intelligence

Ensemble des fonctions mentales ayant pour objet la connaissance conceptuelle et relationnelle.

- Larousse

2. Artificielle

Qui est produit de l'activité humaine (opposé à la nature).

- Larousse

Qu'est-ce que l'intelligence ?

La notion d'intelligence recouvre plusieurs facultés cognitives :

- ① **Raisonnement** : La capacité à résoudre des problèmes et à faire des déductions logiques.
- ② **Apprentissage** : L'aptitude à acquérir de nouvelles connaissances et à s'améliorer grâce à l'expérience.
- ③ **Perception** : La compétence pour reconnaître et interpréter les stimuli sensoriels.
- ④ **Compréhension** : L'habileté à saisir le sens et l'importance de divers concepts et situations.
- ⑤ **Mémorisation** : La faculté de stocker et de rappeler des informations.
- ⑥ **Créativité** : Le pouvoir d'inventer ou de produire de nouvelles idées, de l'originalité dans la pensée.

Mais est-ce que l'intelligence est réductible à des facultés mesurables ?

Conférence de Dartmouth ?

Article

AI Magazine Volume 27 Number 4 (2006) (© AAAI)

A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence

August 31, 1955

John McCarthy, Marvin L. Minsky,
Nathaniel Rochester,
and Claude A. Shannon

■ The 1956 Dartmouth summer research project on artificial intelligence was initiated by this August 31, 1955 proposal, submitted by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon. The original typewritten document of 17 pages plus a title page. Copies of the typewritten are housed in the archives at Dartmouth College and Stanford University. The first 5 pages state the proposal, and the remaining pages give qualifications and interests of the four who proposed the study. In the interest of brevity, this article reproduces only the proposal itself, along with the short autobiographical statements of the proposers.

We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use lan-

guage, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer. The following are some aspects of the artificial intelligence problems.

1. Automatic Computers

If a machine can do a job, then an automatic calculator can be programmed to simulate the machine. The speed and memory capacities of present computers may be insufficient to simulate many of the higher functions of the human brain, but the major obstacle is not lack of machine capacity, but our inability to write programs taking full advantage of what we have.

2. How Can a Computer be Programmed to Use a Language

It may be speculated that a large part of human thought consists of manipulating words according to the rules of meaning and rules of inference. From this point of view, forming a generalization consists of admitting a new

"We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer."

L'intelligence artificielle selon John McCarthy

"It is the science and engineering of making intelligent machines, especially intelligent computer programs. It is related to the similar task of using computers to understand human intelligence, but AI does not have to confine itself to methods that are biologically observable."

— John McCarthy

Le Test de Turing

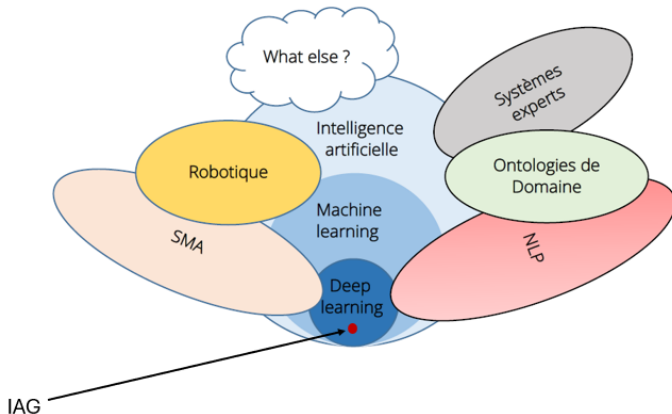
Le Test de Turing, développé par Alan Turing en 1950, est une tentative de mesurer l'intelligence d'une machine, plus précisément de la faculté d'une machine à penser. Cette dernière n'étant pas si évidente à mesurer, le test substitue finalement à la faculté de penser celle de traiter le langage naturel comme un humain.

Les points clés du Test de Turing sont :

- Un interrogateur humain engage une conversation avec un humain et une machine, chacun étant caché de la vue de l'interrogateur.
- Si l'interrogateur ne peut pas déterminer systématiquement quelle est la machine, celle-ci est considérée comme ayant passé le test.
- Le test ne mesure pas la connaissance ou la capacité à être véridique, mais plutôt la capacité de reproduire le comportement humain.

Définition pragmatique de l'intelligence artificielle

Il s'agit d'un ensemble de techniques qui permettent à la machine d'accomplir des tâches qui requièrent traditionnellement une intelligence humaine.



IA forte vs IA faible

La distinction entre IA forte et IA faible se réfère à deux approches conceptuelles différentes dans le domaine de l'intelligence artificielle.

IA faible :

- Aussi connue sous le nom d'IA "étroite", elle est conçue pour effectuer des tâches spécifiques et ne possède pas de conscience.
- Les systèmes d'IA faible agissent et réagissent uniquement en fonction des instructions programmées et des algorithmes spécifiques.
- Exemples : assistants virtuels, systèmes de recommandation, reconnaissance vocale.

IA forte :

- Vise à créer des machines dotées de capacité de généralisation à l'image de l'intelligence humaine.
- L'IA forte serait capable d'apprendre, de raisonner, de résoudre des problèmes et de prendre des décisions indépendamment.
- À ce jour, l'IA forte reste un objectif à atteindre, qui fait l'objet de recherches intensives.

Éléments de machine learning et de deep learning

Qu'est-ce que la machine learning ?

L'apprentissage automatique est une branche de l'intelligence artificielle qui consiste à doter les machines de la capacité d'apprendre à partir de données sans que celles-ci ne soient explicitement programmées pour exécuter des tâches spécifiques.

- **IA discriminative** : Modélise la frontière entre les classes.
Exemples : classification de commentaires clients, routage de mails, credit scoring, diagnostic médical, etc.
- **IA générative** : modélise la distribution des données pour en générer de nouvelles.
Exemples : Génération de texte, d'images, de musique, etc.
- **Apprentissage par renforcement (Reinforcement Learning)** : apprend à prendre des décisions séquentielles dans un environnement.
L'agent n'imité pas ou ne classe pas, il agit pour maximiser une récompense.
Exemples : Algorithmes d'enchères, jeux vidéos, etc.

Machine learning discriminatif

L'apprentissage automatique est une branche de l'intelligence artificielle qui consiste à doter les machines de la capacité d'apprendre à partir de données sans que celles-ci ne soient explicitement programmées pour exécuter des tâches spécifiques.

Le machine learning englobe deux grandes familles d'apprentissage :

- **Supervisé** : Les algorithmes apprennent à partir de données étiquetées pour faire des prédictions ou classifications.
- **Non supervisé** : L'apprentissage est effectué sur des données non étiquetées pour trouver des structures cachées.

L'apprentissage supervisé

L'apprentissage supervisé consiste à apprendre un modèle qui associe une étiquette (*label*) à un ensemble de caractéristiques (*features*).

- **Inputs** : un jeu de données *annotées* pour entraîner le modèle.
 - Exemple : des textes (tweets, etc.) avec les *sentiment* associés, positifs ou négatifs.
- **Output** : une étiquette pour un point de donnée inconnu par le modèle.

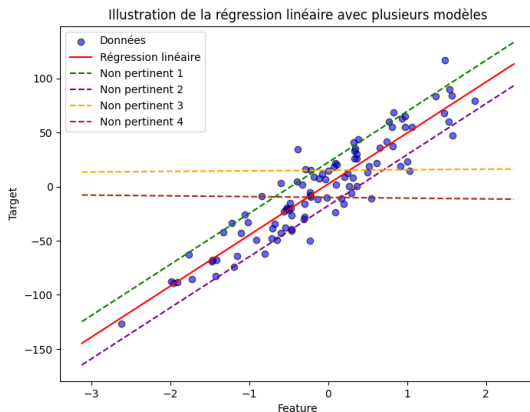
L'apprentissage supervisé se décline lui-même en deux grandes familles :

- **La classification** : prédire une catégorie ou une classe.
 - Exemple : prédire l'étiquette d'une image (chat, chien, etc.), le sentiment associé à un texte, le centre d'intérêt d'un client à partir de ses commentaires, etc.
- **La régression** : prédire une valeur continue (un nombre réel typiquement).
 - Exemple : prédire le prix d'un appartement, la lifetime value d'un client, etc.

Régression linéaire simple

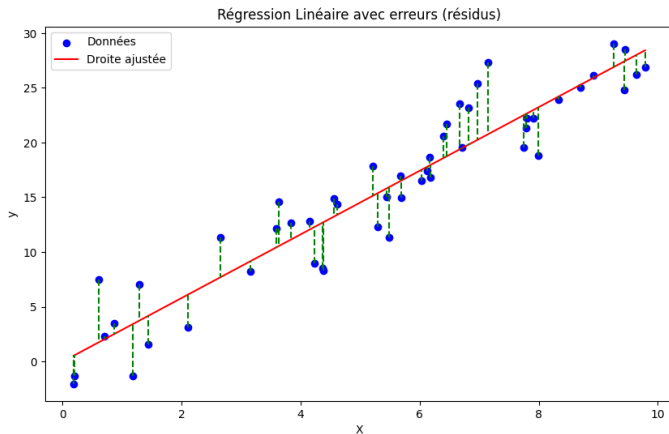
Soit un ensemble de n observations $x_1, x_2, \dots, x_n$ avec les labels correspondants $y_1, y_2, \dots, y_n$, on cherche le modèle linéaire qui ajuste le mieux ces données.

$$\hat{y} = \beta_0 + \beta_1 x$$

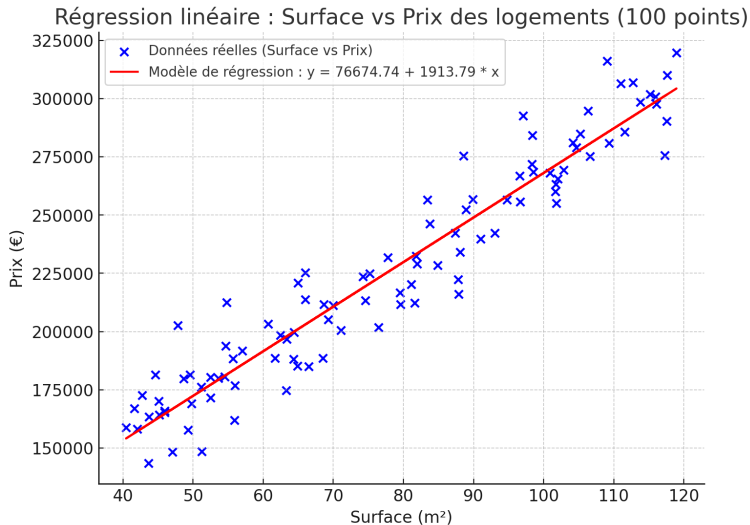


Minimisation du risque empirique 1/2

L'erreur de prédiction pour la i ième observation est : $e_i = y_i - \hat{y}_i$. où $\hat{y}_i = \beta_0 + \beta_1 x_i$.



Prédire le prix en fonction de la surface : inférence



L'apprentissage non supervisé

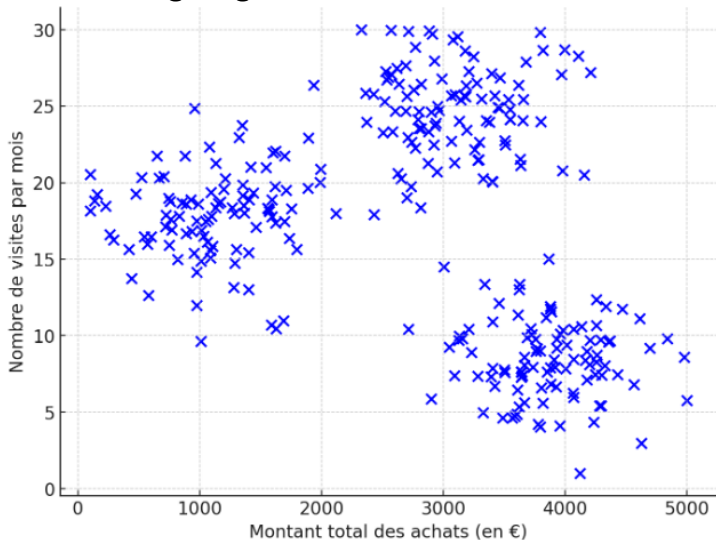
L'apprentissage non supervisé se réfère à l'utilisation de modèles d'apprentissage automatique pour identifier des patterns et des structures dans des données qui ne sont pas étiquetées.

Principales typologies de l'apprentissage non supervisé :

- **Clustering** : Regroupement de points de données similaires ensemble.
Exemple : segmentation de marché, regroupement social.
- **Détection d'anomalies** : Détecter des observations dont les caractéristiques sont inhabituelles par rapport à la majorité.

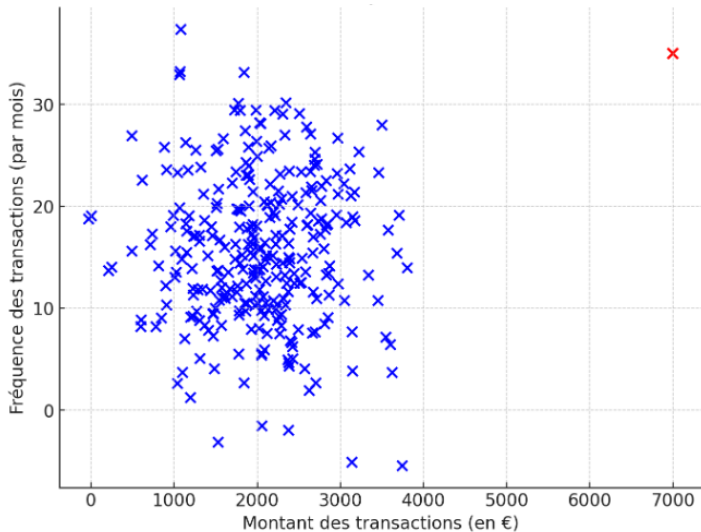
Clustering

Exemple de clustering : segmentation clients



Détection d'anomalies

Exemple de détection d'anomalies : fraude bancaire



Eléments de deep learning

Brève histoire du deep learning

1940-1980 : Fondations

- 1943 : Neurone artificiel (McCulloch & Pitts).
- 1986 : Rétropropagation (Rumelhart, Hinton, Williams).

1990-2000 : Hiver

- Manque de données et puissance de calcul.
- Progrès en théorie : SVM, modèles bayésiens.

2010-2020 : Renaissance

- 2012 : AlexNet révolutionne la vision par ordinateur.

Depuis 2020 : Multimodalité et LLM

- 2017 : Transformer, base des LLM.
- 2018 - 2020 : GPT-3, BERT.
- 2022 : ChatGPT...

Introduction aux réseaux de neurones

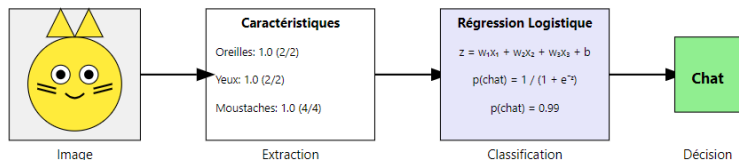
Définition : Les réseaux de neurones sont des modèles computationnels inspirés par le fonctionnement des neurones dans le cerveau humain. Ils sont capables d'apprendre des tâches complexes en modélisant des relations non linéaires entre les entrées et les sorties.

Caractéristiques :

- **Extraction automatique des features** : Capacité d'adaptation et d'extraction des features à partir des données sans programmation explicite.
- **Modélisation non linéaire** : Aptitude à capturer des relations complexes dans les données.
- **Modélisation en grande dimension** : Les modèles de deep learning sont particulièrement adaptés pour les données en grande dimension (images, texte, etc.).
- **Flexibilité** : Applicables à un large éventail de tâches et de typologies de données (images, langage naturel, données graphiques, etc.).

Extraction de features

Extraction de features en machine learning "classique"



Extraction de features en deep learning

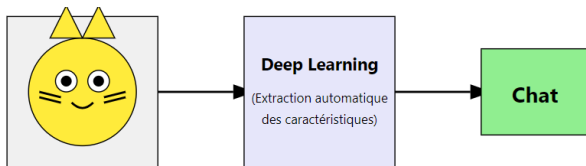
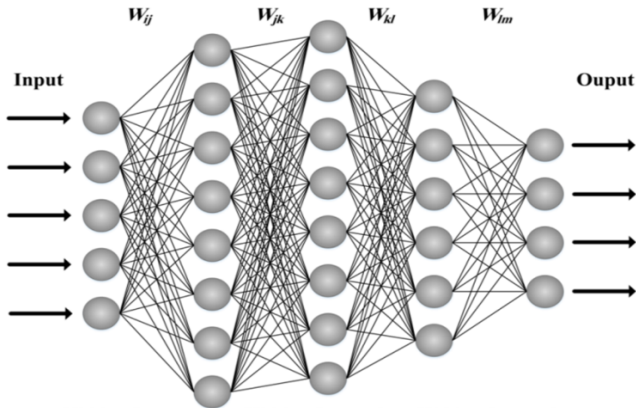


Illustration d'un réseau de neurones classique



Neurone artificiel

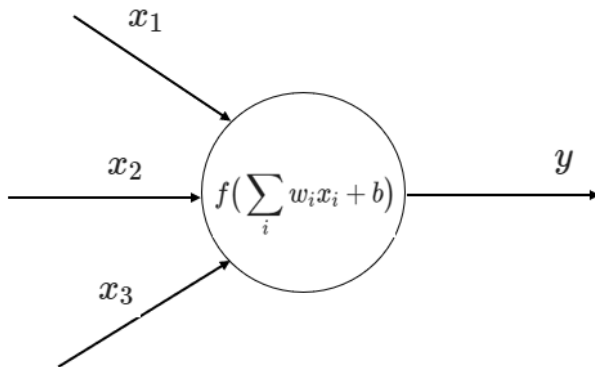
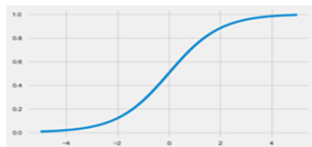
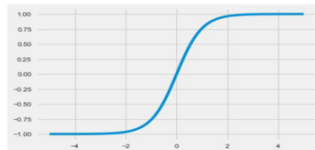


Illustration des fonctions d'activation

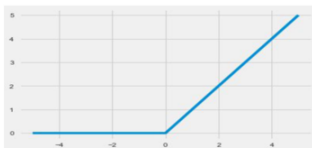
Sigmoïde



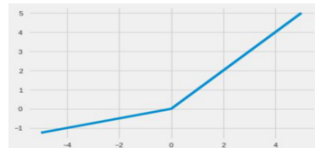
Tanh



ReLU



ReLU paramétrique



La fonction SoftMax dans la classification dans la dernière couche

Le SoftMax transforme une liste de scores en **probabilités de classes**. C'est la dernière étape d'un modèle de classification multiclass.

Formule :

$$P(y_i) = \frac{e^{s_i}}{\sum_{j=1}^K e^{s_j}}$$

où s_i représente le score associé à la classe i , et K le nombre total de classes.

Exemple :

$$\text{Image} \rightarrow \begin{cases} \text{chat} : 0.80 \\ \text{chien} : 0.10 \\ \text{oiseau} : 0.05 \\ \text{lapin} : 0.05 \end{cases}$$

→ Le modèle prédit la classe la plus probable (*chat*), tout en conservant une estimation pour les autres.

Embeddings et représentations distribuées

Introduction aux Embeddings

- **Qu'est-ce qu'un Embedding ?**

- Un embedding est une représentation vectorielle d'un mot qui capture le contexte du mot dans un document, des relations sémantiques et syntaxiques avec d'autres mots.
- Ils transforment des mots en vecteurs de nombres pour que les algorithmes de machine learning puissent les traiter efficacement.

- **Pourquoi sont-ils importants ?**

- Les embeddings permettent de capturer non seulement l'identité d'un mot mais aussi ses aspects sémantiques et contextuels.
- Ils facilitent des tâches telles que la classification de texte, la traduction automatique, et la détection des sentiments.

- **Évolution des embeddings**

- Historiquement, les mots étaient représentés comme des indices ou des vecteurs one-hot, où chaque mot est indépendant des autres.
- Les embeddings modernes, tels que Word2Vec et GloVe, représentent les mots dans des espaces vectoriels continus où les mots de sens similaires sont proches les uns des autres.

Différence entre One-hot Encoding et Embeddings

- **One-hot Encoding**

- Chaque mot est représenté par un vecteur avec un '1' dans la position qui lui est propre et des '0' partout ailleurs. Ce type de représentation est simple mais très inefficace en termes d'espace et ne capture pas les relations entre les mots.
- Exemple : pour un vocabulaire de dimension 100 000 :

Véhicule = $[0, 0, 1, 0, 0, 0, 0, 0, 0, \dots, 0, 0]$

Voiture = $[0, 0, 0, 0, 0, 1, 0, 0, 0, \dots, 0, 0]$

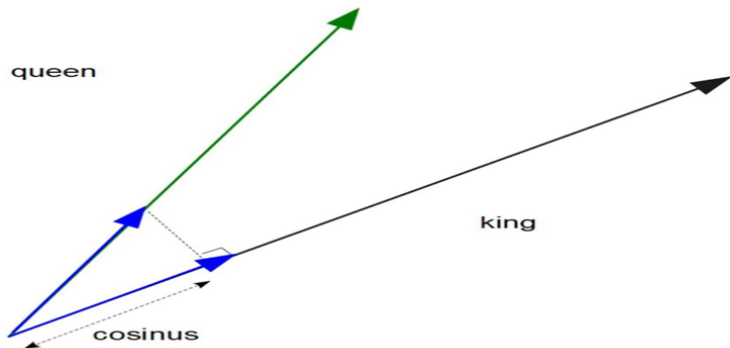
Cette représentation ne permet pas de prendre en compte la dimension sémantique

- **Embeddings**

- Les embeddings représentent les mots comme des vecteurs denses de nombres flottants (généralement entre 50 et 300 dimensions).
- Cette représentation est beaucoup plus riche et peut capturer des relations complexes entre les mots, comme la similarité sémantique.

Similarité sémantique

Nous sommes intéressés par des représentations de mots qui capturent la distance sémantique, sous forme de produit scalaire par exemple (ou distance cosinus).



Représentations distribuées : Principes

"You shall know a word by the company it keeps"

— J.R. Firth (1957)

Les mots sont similaires s'ils apparaissent fréquemment dans le même contexte.

- Il conduit son *véhicule* pour rentrer à la maison.
- Il conduit sa *voiture* pour rentrer chez lui.

Construction d'une représentation distribuée

Considérons le corpus suivant à titre d'exemple : **cnn in crop analysis**

cnn and svm are widely used.

linear_regression performed along with svm

linear_regression for crop and farm

svm being used for farm monitoring

Do cnn, svm and linear_regression appear in the same context?

Le vocabulaire est alors :

[cnn, in, crop, analysis, and, svm, are, widely, used,
linear_regression, performed, along, with, for, farm, being,
monitoring, do, appear, the, same, context]

$$\text{dim}(\text{vocabulaire}) = 22$$

Similarité sémantique

La matrice de co-occurrence représente la fréquence à laquelle les mots apparaissent ensemble deux à deux.

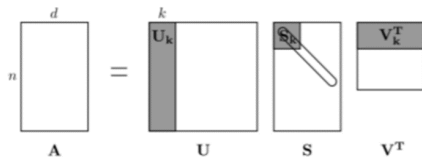
	cnn in crop ...																			
cnn	[0, 2, 1, 1, 1, 2, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1],																			
in	[2, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1],																			
crop	[1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0],																			
...	[1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0],																			
	[2, 1, 0, 0, 1, 0, 1, 1, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],																			
	[1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],																			
	[1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],																			
	[1, 0, 0, 0, 1, 2, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0],																			
	[1, 1, 1, 0, 1, 2, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1],																			
	[0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0],																			
	[0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0],																			
	[0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0],																			
	[0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 2, 1, 1, 0, 0, 0, 0],																			
	[0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 2, 0, 1, 1, 0, 0, 0, 0],																			
	[0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0],																			
	[0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0],																			
	[1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1],																			
	[1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1],																			
	[1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1],																			
	[1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0],																			
	[1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0]																			

Réduction de la dimension

La décomposition en valeurs singulières est une technique permettant de réduire la dimension des données (similaire à l'ACP).

Étant donné une matrice $A \in \mathbb{R}^{n \times d}$

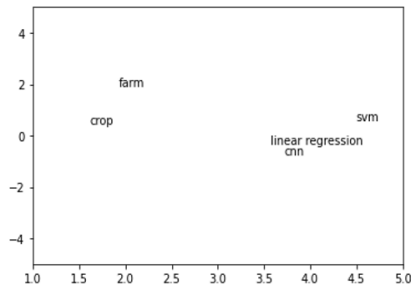
$$A = UDV^T \quad \text{où} \quad U \in \mathbb{R}^{n \times r}, D \in \mathbb{R}^{r \times r}, V \in \mathbb{R}^{d \times r}.$$



U est la matrice contenant les représentations (vecteurs de mots)

Visualisation des représentations distribuées

```
cnn [ 3.72113518, -0.73233585],  
ln [ 2.7940387 , -1.40864784],  
crop [ 1.61178082,  0.44786021],  
... [ 0.77784994, -0.31230389],  
[ 2.12245048,  1.29571556],  
[ 4.49293777,  0.58090417],  
[ 1.33312748,  0.66758743],  
[ 1.33312748,  0.66758743],  
[ 2.25882809,  1.80738544],  
[ 3.56593605, -0.31006976],  
[ 0.95394179,  0.079159 ],  
[ 0.95394179,  0.079159 ],  
[ 0.95394179,  0.079159 ],  
[ 1.92921553,  1.94783497],  
[ 1.92921553,  1.94783497],  
[ 1.12301312,  1.42126096],  
[ 1.12301312,  1.42126096],  
[ 2.26025282, -1.31571175],  
[ 2.26025282, -1.31571175],  
[ 2.26025282, -1.31571175],  
[ 2.26025282, -1.31571175],  
[ 2.26025282, -1.31571175]
```



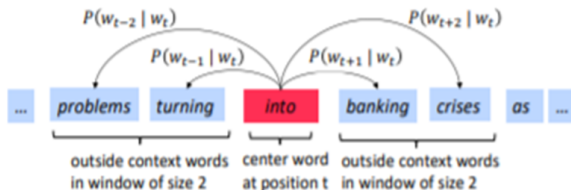
Word2Vec : Principes

- **Principes de base :**

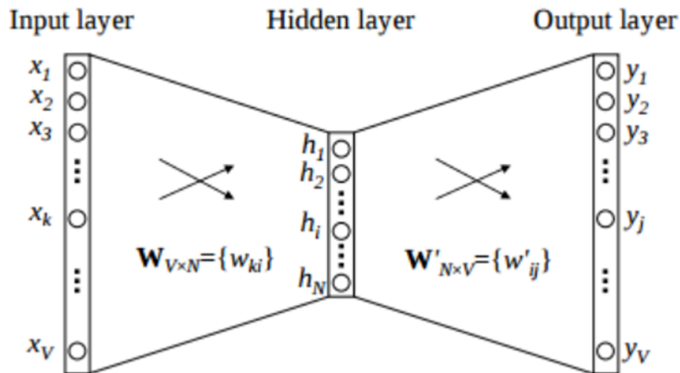
- Word2Vec est basé sur l'hypothèse distributionnelle : les mots qui apparaissent dans des contextes similaires ont des significations similaires.
- Utilise un réseau de neurones peu profond pour apprendre des embeddings de mots à partir de grands corpus.

- **Deux architectures principales :**

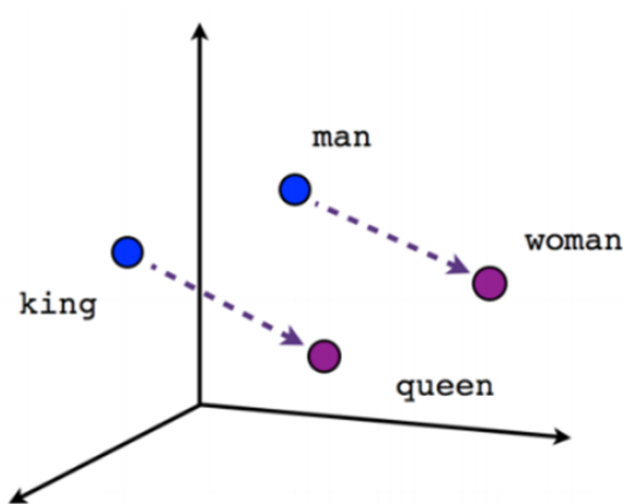
- 1 *Continuous Bag of Words (CBOW)* : Prédit le mot cible à partir du contexte.
- 2 *Skip-Gram* : Prédit le contexte à partir du mot cible.



Architecture du modèle CBOW



Représentations Word2Vec



Applications pratiques de Word2Vec

- **Analogie de mots :**

- Word2Vec est célèbre pour capturer des relations complexes, comme "homme est à femme ce que roi est à reine".
- Permet de résoudre des analogies en utilisant des opérations arithmétiques simples sur les vecteurs de mots.

- **Clustering sémantique :**

- Les embeddings peuvent être utilisés pour regrouper des mots sémantiquement similaires, facilitant l'analyse de textes volumineux.

- **Amélioration des systèmes de recommandation :**

- Les vecteurs de mots de Word2Vec peuvent être utilisés pour améliorer la précision des recommandations en comprenant mieux les préférences des utilisateurs.

Autres représentations distribuées

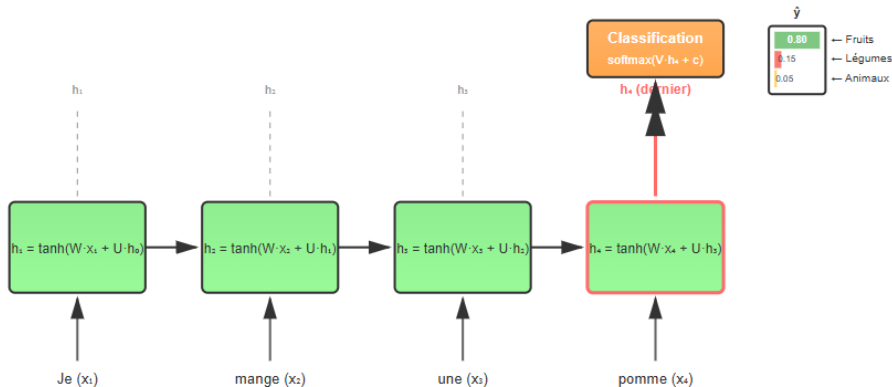
Il existe d'autres représentations distribuées :

- Glove (2014) : proposé par une équipe de Stanford, il combine à la fois les techniques modernes de deep learning et les techniques statistiques (co-occurrences entre les mots). Il permet d'avoir des représentations des mots plus globales que Word2Vec.
- Fasttext (à partir de 2016) : la librairie a été créée par Facebook. Le modèle est entraîné sur des subwords (n-grams de caractères). Il est ainsi plus efficace pour traiter les mots inconnus (out of vocabulary).
- Représentations contextuelles (Transformers), etc.

Large Language Models (LLMs)

Anciens modèles pour le traitement du langage : RNN

- **Mémoire à court terme** : Les réseaux de neurones récurrents (RNN) ont la capacité de se "souvenir" d'informations passées grâce à leurs connexions récurrentes.
- **Traitement de séquences** : Ils sont particulièrement adaptés pour des tâches où les données sont séquentielles et où le contexte est important.



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukas Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukas and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

Transformer originel

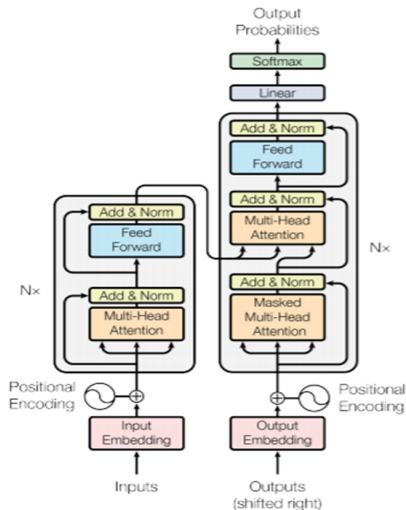
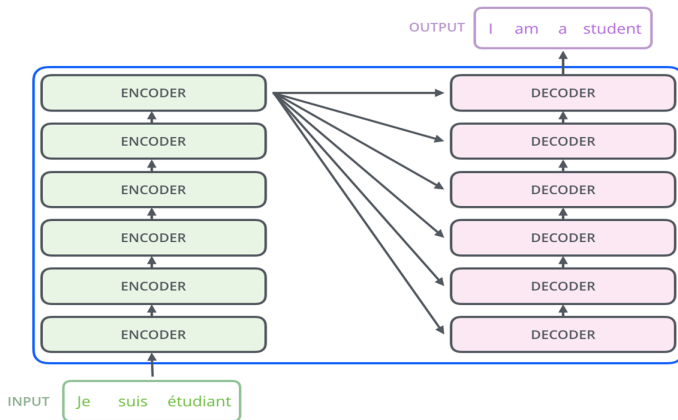
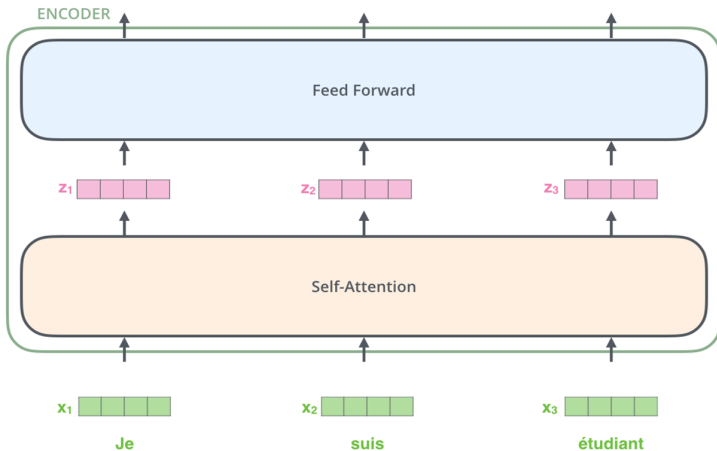


Figure 1: The Transformer - model architecture.

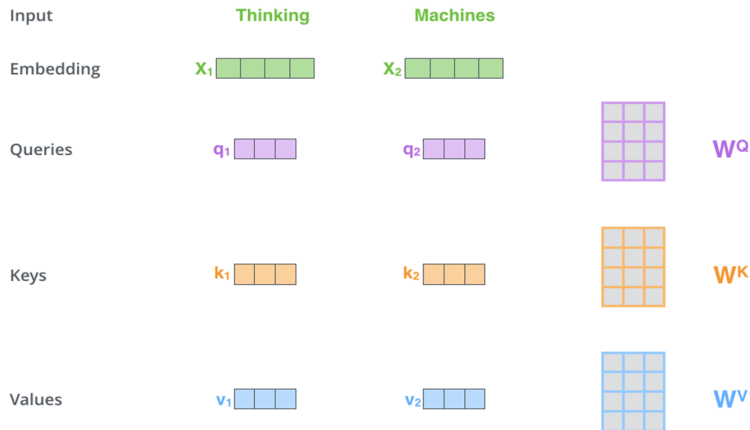
Mécanisme de cross-attention



Mécanisme de self-attention



Fonctionnement du mécanisme de self-attention



Calcul des scores de self-attention

Input

Embedding

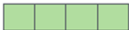
Queries

Keys

Values

Score

Thinking

x_1 

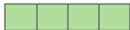
q_1 

k_1 

v_1 

$$q_1 \cdot k_1 = 112$$

Machines

x_2 

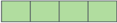
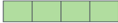
q_2 

k_2 

v_2 

$$q_1 \cdot k_2 = 96$$

Calcul des scores de self-attention

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_2 \cdot k_2 = 96$
Divide by $8 (\sqrt{d_k})$	14	12
Softmax	0.88	0.12

Calcul de la nouvelle représentation

Input

Embedding

Queries

Keys

Values

Score

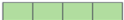
Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax
X
Value

Sum

Thinking

x_1 

q_1 

k_1 

v_1 

$q_1 \cdot k_1 = 112$

14

0.88

v_1 

z_1 

Machines

x_2 

q_2 

k_2 

v_2 

$q_1 \cdot k_2 = 96$

12

0.12

v_2 

z_2 

Performances du tranformer originel

Entraîné sur WMT 2014 English- German dataset, comprenant près de 4.5 millions de phrases sentence, le WMT 2014 English-French dataset, comprenant près de 36 millions de phrases.

- 8 NVIDIA P10 GPUs
- **Base** : 12 heures
- **Large** : 3.5 jours

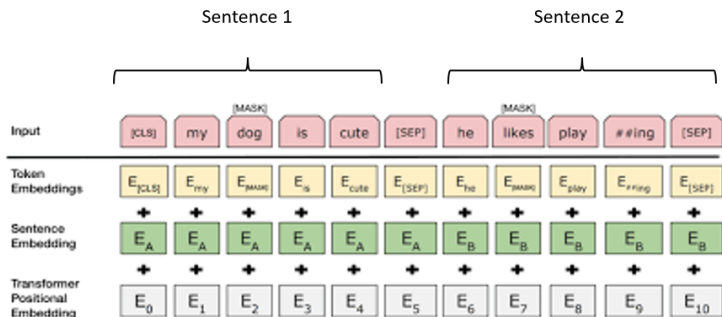
Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

BERT

BERT (Bidirectional Encoder Representations from Transformers) représente une avancée majeure dans le NLP, introduisant une approche novatrice pour la modélisation de langage. Il utilise la bidirectionnalité pour comprendre le contexte des mots, permettant une compréhension plus fine du langage.

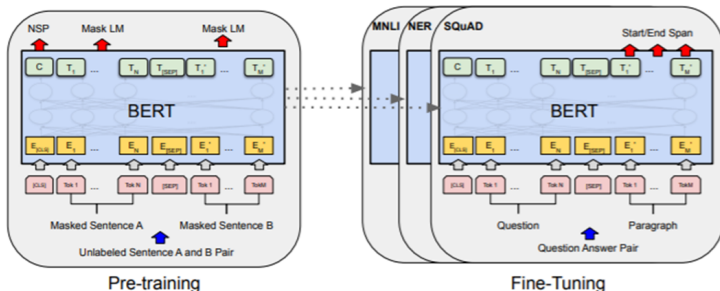
Pré-entraînement



Fine-tuning de BERT pour des tâches spécifiques

Après pré-entraînement, BERT est affiné pour des tâches spécifiques:

- Ajout d'une couche de sortie spécifique à la tâche (classification, NER, QA).
- Fine-tuning de tous les paramètres du modèle pré-entraîné sur le corpus de la tâche.



Performances de BERT

BERT a été pré-entraîné sur le BookCorpus (800 millions de mots) et English Wikipedia (2,500 millions de mots).

Deux modèles :

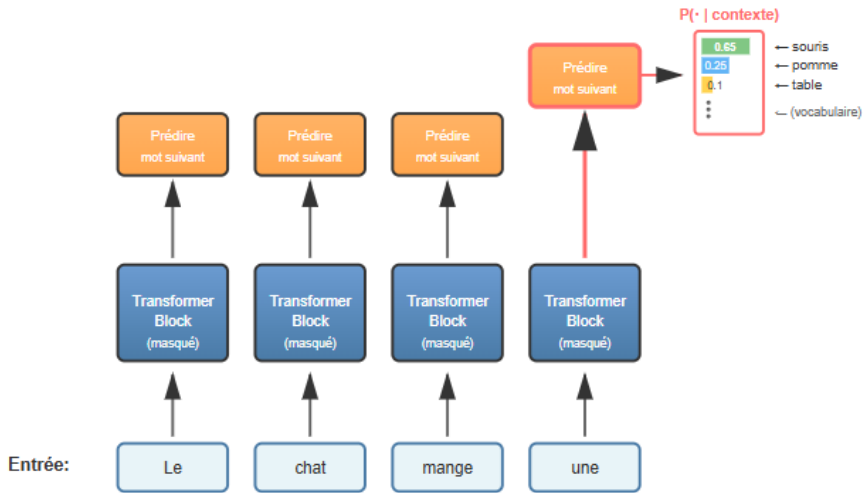
- **BERT Base** : 12 couches avec 110 millions de paramètres.
- **BERT Large** : 24 couches avec 340 millions de paramètres.

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

*« Ce que je ne peux pas créer,
je ne le comprends pas. »*

– Richard Feynman

Fonctionnement des modèles génératifs



Le paramètre de température

Définition La température (T) contrôle le degré d'aléa dans les réponses générées. Elle agit sur la distribution de probabilité des mots avant l'échantillonnage.

Formule mathématique :

$$P(x_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

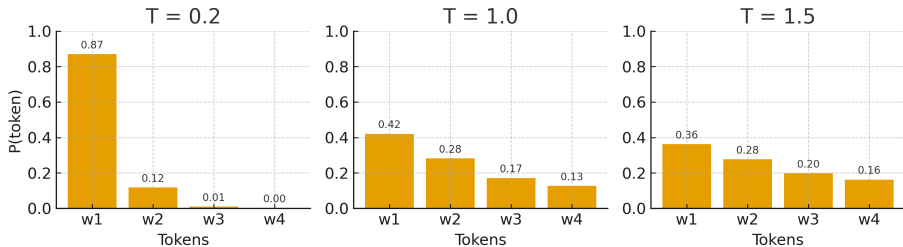
- $T < 1 \rightarrow$ distribution plus concentrée, réponses plus prévisibles
- $T > 1 \rightarrow$ distribution plus plate, réponses plus variées et créatives

Autrement dit :

- Basse température $\rightarrow$ précision et cohérence.
- Haute température $\rightarrow$ imagination et diversité.

Illustration graphique de l'effet de la température

Effet de la température sur la distribution des probabilités



La tokenisation

Problème de départ : Les modèles de langage ne peuvent pas traiter directement du texte brut : les mots, les accents, la ponctuation et les variations sont trop nombreux (plusieurs millions de combinaisons possibles).

La tokenisation : Elle consiste à découper le texte en **unités réutilisables et limitées en nombre**, appelées *tokens*. Chaque token est ensuite représenté par un identifiant numérique que le modèle peut apprendre à prédire.

Intérêt de la tokenisation :

- Réduire la complexité du vocabulaire : on passe de millions de mots à quelques dizaines de milliers de tokens.
- Gérer les mots inconnus ou rares en les décomposant en sous-unités connues.
- Offrir une représentation numérique uniforme du langage, compatible avec les réseaux de neurones.

La composition des données d'entraînement des modèles

Principe général Les modèles de langage sont entraînés sur d'immenses corpus de texte — souvent plusieurs milliers de milliards de tokens — issus de sources très diverses pour couvrir la langue, les faits et les styles.

Sources typiques de données :

- **Web public** : pages web, articles, forums, Wikipédia (ex. Common Crawl).
- **Livres et documents** : œuvres littéraires, articles scientifiques, rapports techniques.
- **Données de code** : dépôts GitHub, notebooks, documentation technique.
- **Conversations et dialogues** : jeux de données de chat, discussions de forums, données annotées par des humains.
- **Données synthétiques** : textes générés par d'autres modèles pour enrichir certaines thématiques.

Équilibre recherché :

- Diversité linguistique et thématique (langues, registres, domaines).
- Qualité et cohérence (filtrage du spam, des doublons, des contenus toxiques).
- Représentation équilibrée entre langage courant, technique et conversationnel.

GPT-3

- Développé par OpenAI, publié en 2020.
- Modèle auto-régressif basé sur l'architecture Transformer.

Caractéristique	Valeur / Détail
Date	2020
Paramètres	175 milliards
Corpus d'entraînement	Environ 300 milliards de tokens
Contexte maximum	2048 tokens
Ressources GPU	Plusieurs centaines de GPU (type V100)
Coût d'entraînement (estim.)	4,6 M\$ à 12 M\$ (selon les sources)

- **Applications** : génération de texte, Q&R, résumé, traduction, assistance à la programmation.

Le few-shot learning

- Les grands modèles (comme GPT-3) peuvent réaliser des tâches **sans être explicitement réentraînés** dessus.
- Le principe repose sur **l'utilisation de quelques exemples** (exemples étiquetés) directement dans le prompt ou l'entrée.
- **Zero-shot** : aucune démonstration fournie, le modèle doit comprendre la tâche à partir de sa connaissance générale.
- **One-shot / Few-shot** : on inclut un nombre très réduit d'exemples (un ou quelques-uns) pour guider le modèle dans la résolution de la tâche.
- Exemple :

English: "cat" → French: "chat"

English: "house" → French: "maison"

English: "car" → French: "voiture"

English: "tree" → French:

ChatGPT

ChatGPT, développé par OpenAI, est un modèle de langage basé sur l'architecture GPT (Generative Pre-trained Transformer) optimisé pour comprendre et générer des dialogues naturels. L'entraînement de ChatGPT se décline selon les étapes

suivantes :

- ➊ **Pré-entraînement sur un corpus volumineux** : Comme GPT-3, ChatGPT est d'abord pré-entraîné sur un vaste ensemble de données textuelles, englobant un large éventail de la littérature disponible sur Internet, pour apprendre une compréhension générale du langage.
- ➋ **Fine-tuning supervisé** : Ensuite, ChatGPT est affiné sur des dialogues spécifiques pour améliorer ses compétences conversationnelles. Cette étape utilise des paires question-réponse et des conversations pour enseigner au modèle des structures de dialogue et des réponses contextuellement appropriées.
- ➌ **Reinforcement Learning from Human Feedback (RLHF)** : Utilisation de techniques de renforcement pour ajuster les réponses du modèle basées sur les préférences et les corrections fournies par des évaluateurs humains, raffinant davantage la pertinence et la naturalité des réponses.

LLMs propriétaires vs Open Source

● Modèles propriétaires

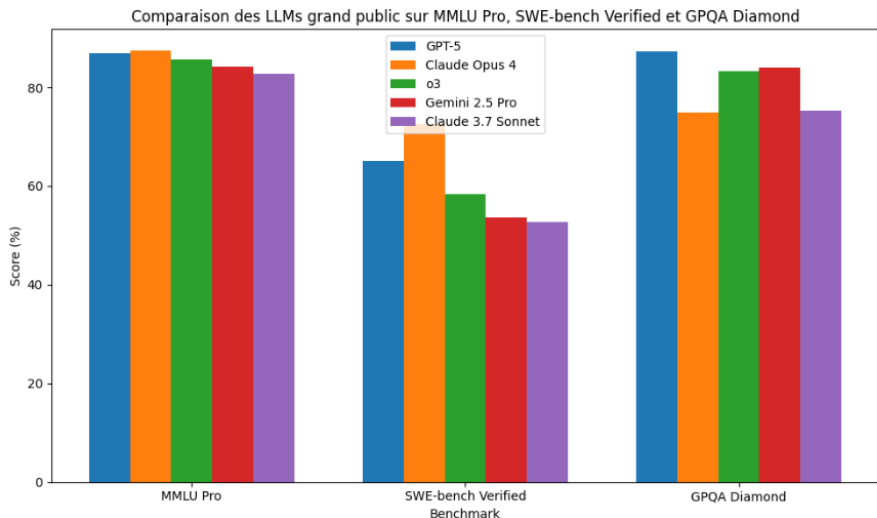
- ChatGPT, Claude, Gemini, Grok...
- Poids non disponibles au public.
- Performances élevées, accès limité par API payante
- Protection intellectuelle stricte (code non accessible)
- Support technique assuré, documentation avancée
- Dépendance aux fournisseurs et coûts potentiellement élevés

● Modèles open source

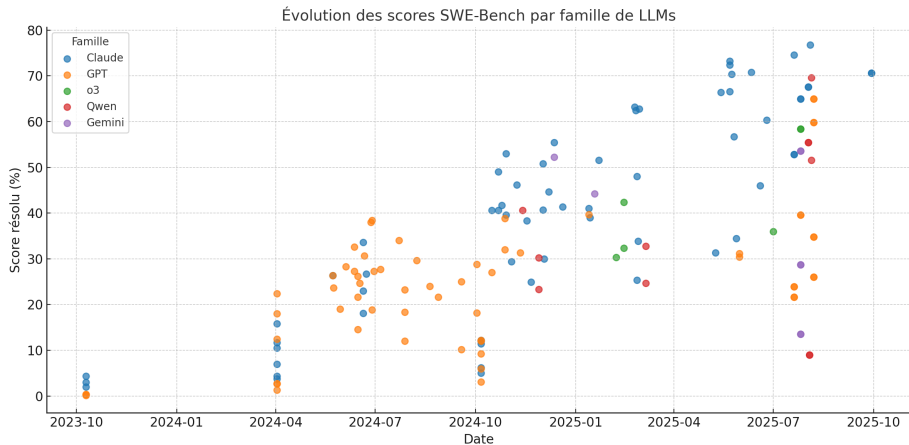
- Llama, Mistral, Gemma, DeepSeek, Qwen...
- Poids disponibles au public
- Transparence du code, flexibilité d'adaptation
- Performances parfois inférieures, mais amélioration continue
- Gratuit, mais coût de l'infrastructure à gérer
- Exposition aux failles potentielles de sécurité, support communautaire

Conclusion : Choisir entre contrôle, flexibilité et performances optimales.

Comparaison des performances des LLMs : benchmarks classiques



Evolution des performances des LLMs sur SWE-Bench



Introduction au prompting

Définition : Le prompting est l'art de formuler des instructions à un modèle de langage afin d'obtenir une réponse pertinente et de qualité.

Idée clé : La qualité de la sortie dépend fortement de la manière dont l'entrée est rédigée.

Enjeux :

- Obtenir des résultats précis et cohérents.
- Réduire les risques d'ambiguïtés et d'hallucinations.
- Adapter le style et le niveau de détail de la réponse.

Techniques de base du prompting

- **Prompt clair et explicite** : éviter les formulations vagues.
- **Contexte** : fournir des informations supplémentaires pour orienter le modèle.
- **Format attendu** : indiquer la structure de sortie souhaitée (liste, tableau, texte court).
- **Exemples** : montrer un ou plusieurs cas pour guider la génération.

Exemple : Au lieu de "C'est quoi le machine learning ?", écrire "Donne une explication détaillée du fonctionnement du machine learning, illustrée avec un exemple".

1. Le ton change la structure du raisonnement

Principe

Le ton du prompt influence non seulement le style, mais aussi la structure logique de la réponse.

Effets observés

- Ton calme et analytique : produit une écriture déductive et ordonnée, où le modèle explicite les causes et développe les idées avant de conclure.
- Ton urgent ou tendu : génère une écriture télégraphique, orientée vers l'action, avec phrases courtes et listes de décisions.

Exemples

« Explique posément, en suivant un raisonnement logique. » « Vite : résume-moi ce que je dois dire au client dans deux minutes. »

À retenir

Le ton est un levier cognitif : il modifie la manière dont le modèle raisonne avant même d'écrire.

2. Les métaphores encadrent la pensée

Principe

La métaphore impose un cadre de raisonnement : elle force le modèle à réencoder le concept dans une structure familière.

Effets observés

- Elle permet d'aborder des notions abstraites par transfert analogique.
- Elle structure naturellement la réponse selon les dimensions de la métaphore (par exemple : ingrédients, recettes, cuisson).

Exemples

« Explique le machine learning comme si c'était une cuisine. » « Explique l'optimisation comme une randonnée avec des étapes et des sommets. »

À retenir

La métaphore agit comme un cadre cognitif qui oriente la logique et le vocabulaire du texte.

3. Les rôles activent des corpus latents

Principe

Attribuer un rôle au modèle sélectionne une manière de parler et de raisonner issue de son corpus.

Effets observés

- Historien : met l'accent sur la chronologie et les causes profondes.
- Ingénieur sceptique : insiste sur les hypothèses, les tests et les limites.
- Poète critique : privilégie les images, les tensions et le rythme.

Exemples

« Agis comme un historien en expliquant les causes profondes. » « Réponds comme un ingénieur qui doute de ses propres hypothèses. »

À retenir

Le rôle agit comme un filtre stylistique et culturel qui oriente la réponse sans en changer le fond.

4. La précision du contexte vaut plus que la longueur

Principe

Un contexte précis et incarné produit de meilleures réponses qu'une longue liste d'instructions abstraites.

Effets observés

- Le modèle comprend mieux la situation lorsqu'elle est ancrée dans un cadre concret.
- Les réponses deviennent plus pertinentes et plus naturelles.

Exemples

« Tu écris à ton supérieur, ton ferme mais respectueux, objectif : obtenir une validation avant 18 h. » « En 200 mots, identifie trois risques et formule une demande de décision. »

À retenir

Un cadre incarné aide le modèle à raisonner comme dans une situation réelle.

5. Les verbes d'action changent la dynamique

Principe

Le verbe choisi détermine le type de raisonnement attendu.

Effets observés

- Explique : logique de cause à effet.
- Montre : logique d'exemple et d'illustration.
- Révèle : logique d'enquête ou de découverte.

Exemples

« Explique en trois causes principales. » « Montre deux exemples concrets et un contre-exemple. » « Révèle l'élément caché qui change la conclusion. »

À retenir

Le verbe oriente la structure du texte et la nature du raisonnement.

6. Le silence comme outil

Principe

Ne pas tout préciser laisse au modèle une marge d'interprétation, propice à la créativité.

Effets observés

- Le modèle complète avec des régularités contextuelles, souvent pertinentes.
- Trop de contraintes bloque la pensée et conduit à des réponses formatées.

Exemples

« Donne trois options, dont une plus audacieuse, sans les justifier. » « Rédige un brouillon brut, sans titres, avec les idées clés uniquement. »

À retenir

Le vide agit comme un espace de respiration qui stimule la créativité.

7. Les émotions modulent l'énergie du contenu

Principe

Le ton émotionnel du prompt influence le rythme et la profondeur de la réponse.

Effets observés

- Colère : texte direct, tranchant, orienté vers la critique.
- Tristesse : rythme lent, ton grave, introspection.
- Joie : rythme rapide, style fluide et enthousiaste.
- Admiration : langage figuré et ample.

Exemples

« Sans détour, dis-moi ce qui ne va pas dans cette stratégie. » « Parle avec gravité des risques systémiques en trois points. »

À retenir

L'émotion structure le raisonnement en modulant l'énergie et la focalisation.

8. Le paradoxe pousse la nuance

Principe

Introduire une contradiction dans le prompt oblige le modèle à produire une réponse nuancée.

Effets observés

- Le modèle cherche un équilibre entre deux pôles logiques.
- Il produit une pensée dialectique plutôt qu'un simple commentaire.

Exemples

« Explique pourquoi l'IA est à la fois une menace et une promesse. » « Montre en quoi le progrès peut être à la fois un risque et une chance. »

À retenir

Le paradoxe favorise la nuance et la réflexion véritable.

Prompting pour le code

Le prompt ne doit pas être considéré comme une question, mais une spécification logicielle implicite.

Un LLM :

- ne découvre pas le problème
- n'infère pas correctement les contraintes manquantes
- optimise ce qui est explicitement formulé

Conclusion :

- prompt vague $\Rightarrow$ code instable
- prompt précis $\Rightarrow$ comportement déterministe

Structure d'un prompt robuste

Un prompt de code doit contenir explicitement :

- ❶ Rôle technique
- ❷ Objectif final
- ❸ Contraintes
- ❹ Interdits
- ❺ Exemples
- ❻ Critère de qualité dominant

Omission d'un élément conduit à un arbitrage implicite du modèle LLM utilisé pour générer le code.

Rôle et orientation du modèle

Le rôle oriente le modèle.

Exemples :

- ingénieur junior $\Rightarrow$ code verbeux, peu abstrait
- ingénieur senior $\Rightarrow$ factoring, invariants implicites
- ingénieur production $\Rightarrow$ gestion des erreurs, robustesse

Utile à préciser :

- seniorité
- contexte (production, POC, teaching)
- culture (clean code, pragmatique, performance)

Objectif : ce que le code optimise réellement

Le modèle tend toujours vers un objectif.

Objectifs possibles :

- lisibilité
- maintenabilité
- performance
- rapidité d'implémentation
- facilité de test

Sans objectif explicite :

- le modèle sur-optimise la généralité
- complexifie inutilement

Contraintes techniques explicites

Les contraintes réduisent l'espace de recherche.

Utile à préciser :

- langage + version exacte
- taille et nature des données
- dépendances autorisées ou non
- contraintes de performance ou mémoire

Un LLM sans contraintes va tendre à agir comme un générateur de patterns génériques.

Modèles de raisonnement

Objectif : améliorer la capacité des LLMs à résoudre des problèmes complexes en rendant explicites les étapes de raisonnement.

Principe des Chain-of-Thoughts (CoT) :

- Décomposer une question en **étapes intermédiaires logiques**.
- Forcer le modèle à “**penser à voix haute**” avant de donner la réponse finale.
- Exemple : résoudre une équation, planifier une suite d'actions, raisonner sur des données tabulaires.

Méthodes d'entraînement :

- **Fine-tuning supervisé** sur des datasets annotés avec étapes de raisonnement.
- **Self-consistency** : générer plusieurs chaînes et agréger la réponse la plus fréquente/cohérente.
- **Distillation** : transférer les capacités de raisonnement d'un grand modèle vers un plus petit.

Limites actuelles des LLMs : deux axes majeurs

1. Hallucinations

- Génération de contenus inexacts ou inventés, parfois avec une forte confiance.
- Difficulté à distinguer le vrai du plausible, surtout hors du domaine d'entraînement.
- Conséquences : risque de désinformation, perte de fiabilité, nécessité de vérification humaine.

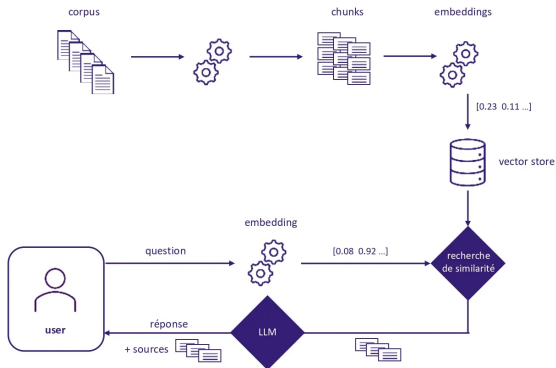
2. Non-déterminisme

- Un même prompt peut produire plusieurs réponses différentes.
- Résultats sensibles à la température, au contexte ou à de légères variations de formulation.
- Enjeu pour la reproductibilité et le contrôle qualité des systèmes fondés sur les LLMs.

Retrieval Augmented Generation (RAG)

Principes du RAG

RAG basique

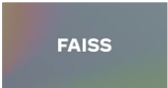


Stratégies de chunking

- **Définition** : Le chunking consiste à segmenter un document en morceaux (chunks) pour optimiser la récupération et la génération d'informations dans un système RAG.
- **Principales stratégies** :
 - **Fixed-size chunking** : découpage en segments de longueur fixe (ex : 512 tokens), simple mais peut fragmenter le contexte.
 - **Overlapping chunks** : chevauchement entre les segments pour conserver le contexte et éviter la perte d'informations critiques.
 - **Semantic chunking** : segmentation basée sur la structure du texte (paragraphe, titres, sections) ou l'analyse sémantique.
 - **Recursive chunking** : division progressive en fonction de la granularité des informations pertinentes.
- **Exemples d'outils** : LangChain, NLTK, SpaCy, tiktoken.
- **Avantages** : Améliore la récupération d'informations, réduit le bruit et optimise l'usage des modèles génératifs.

Bases de données vectorielles

- **Définition** : Stockent et indexent des représentations numériques (embeddings) pour effectuer des recherches sémantiques efficaces.
- **Principe** : Chaque document est transformé en un vecteur dense dans un espace de grande dimension. La recherche repose sur la similarité (cosinus, ANN, etc.) plutôt que sur des mots-clés.

The FAISS logo is a rectangular box with a gradient background transitioning from purple to green, with the word "FAISS" in white, bold, sans-serif font centered inside.

FAISS

Approches avancées du RAG

- **Reranking** : Amélioration de la sélection des documents récupérés en utilisant un modèle plus puissant (ex : cross-encoder, Cohere Rerank, ColBERT).
- **Fine-tuning du retrieveur** : Optimisation de l'indexation et du scoring des documents avec un modèle entraîné spécifiquement sur le domaine d'application.
- **Graph RAG** : Utilisation d'un graphe de connaissances pour structurer les relations entre documents et améliorer la récupération d'information.
- **Agent RAG** : Intégration d'agents conversationnels capables d'exécuter des actions (recherches multiples, vérification des réponses) avant de générer une réponse finale.

Reranking dans le RAG

- **Définition** : Le reranking consiste à réordonner les documents récupérés pour améliorer la pertinence des résultats avant de les passer au modèle génératif.
- **Méthodes principales** :
 - **BM25 + Reranker** : recherche lexicale suivie d'un modèle neuronal qui affine le classement des documents.
 - **Cross-encoder** : modèle BERT-like qui évalue chaque document en prenant en compte à la fois la requête et le contenu.
 - **Dense Reranking** : ajuste les scores des documents en utilisant des embeddings appris sur des données spécifiques.
- **Exemples d'outils** : Cohere Rerank, ColBERT, MonoT5, Rank-BERT.
- **Avantages** : Meilleure précision des résultats, réduction du bruit, amélioration des performances du modèle génératif.

Fine-tuning dans le RAG

- **Définition** : Le fine-tuning consiste à ajuster un modèle pré-entraîné sur un ensemble de données spécifique pour améliorer ses performances dans un domaine précis.
- **Types de fine-tuning** :
 - **Fine-tuning du retrieveur** : amélioration de la récupération des documents en entraînant un modèle dense (ex : Contriever, ColBERT) sur un corpus spécialisé.
 - **Fine-tuning du modèle génératif** : adaptation d'un LLM pour mieux exploiter les documents récupérés et générer des réponses plus pertinentes.
 - **Fine-tuning hybride** : entraînement simultané du retrieveur et du modèle génératif pour une synergie optimale.
- **Avantages** : L'avantage principal est une meilleure spécialisation du modèle.

Graph RAG

- **Définition** : Le Graph RAG utilise une structure de graphe pour organiser et récupérer l'information de manière plus contextuelle et structurée.
- **Principes clés** :
 - **Relations entre documents** : les nœuds représentent des documents, concepts ou entités, et les arêtes définissent leurs connexions.
 - **Recherche basée sur la connectivité** : la récupération d'informations se fait en explorant les relations entre les nœuds pour trouver les réponses les plus pertinentes.
 - **Augmentation du contexte** : le modèle peut utiliser des chemins sémantiques dans le graphe pour enrichir le prompt avec des données plus cohérentes.
- **Exemples d'outils** : Neo4j, NetworkX, Weaviate.
- **Avantages** : Meilleure structuration des connaissances, récupération d'information plus précise, réduction des erreurs contextuelles.

Agent RAG

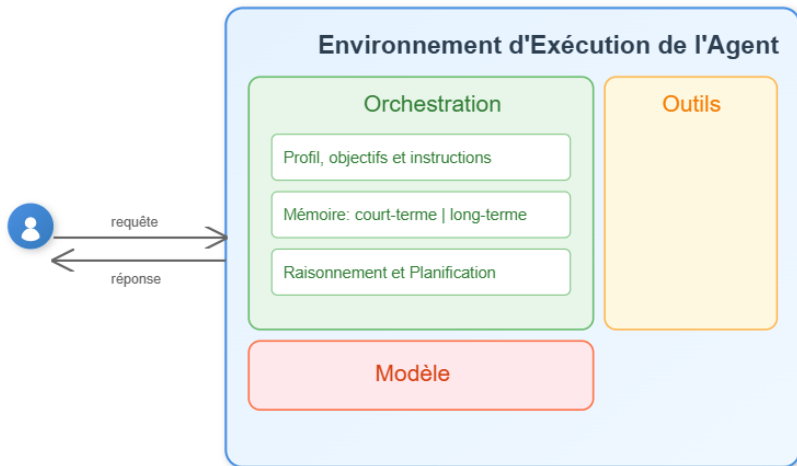
- **Définition** : L'Agent RAG intègre un agent autonome qui orchestre la récupération et l'exploitation des documents pour affiner la génération de réponses.
- **Principes clés** :
 - **Multi-recherches adaptatives** : l'agent effectue plusieurs requêtes en fonction du contexte et ajuste dynamiquement le retrieveur.
 - **Vérification et correction** : l'agent peut analyser la réponse générée, détecter d'éventuelles erreurs et reformuler la requête si nécessaire.
 - **Actions spécifiques** : l'agent peut interagir avec des bases de données, des API externes ou exécuter du code pour enrichir la réponse finale.
- **Exemples d'outils** : LangChain Agents, CrewAI, etc.
- **Avantages** : Plus grande autonomie, récupération d'informations plus dynamique, réduction des erreurs génératives.

Agents basés sur les LLM

Introduction aux Agents

- **Définition** : Les agents basés sur les LLMs sont des systèmes capables de prendre des actions de manière autonome en utilisant des outils mis à leur disposition.
- **Pourquoi les agents ?**
Les modèles de langage seuls sont limités par leurs données d'entraînement. Les agents étendent ces capacités en utilisant des outils externes.
- **Objectif de la présentation** :
 - Expliquer le rôle des agents en IA.
 - Présenter leur architecture et leurs composants.
 - Explorer les cas d'utilisation et les techniques modernes d'orchestration.

Principe d'un Agent



Chain-of-Thought (CoT)

- **Qu'est-ce que CoT ?**

Une technique de raisonnement permettant aux modèles de langage de diviser une tâche complexe en étapes intermédiaires.

- **Fonctionnement :**

- Le modèle génère une séquence de raisonnements explicites avant de fournir une réponse.
- Permet d'améliorer la précision et la transparence des décisions prises par un agent.

- **Applications :**

- Résolution de problèmes mathématiques complexes.
- Compréhension de texte avec inférences logiques.
- Réponses plus précises et cohérentes dans les dialogues IA.

ReAct : Reasoning + Acting

Définition : ReAct est une approche qui combine le raisonnement et l'action pour améliorer la prise de décision des modèles de langage.

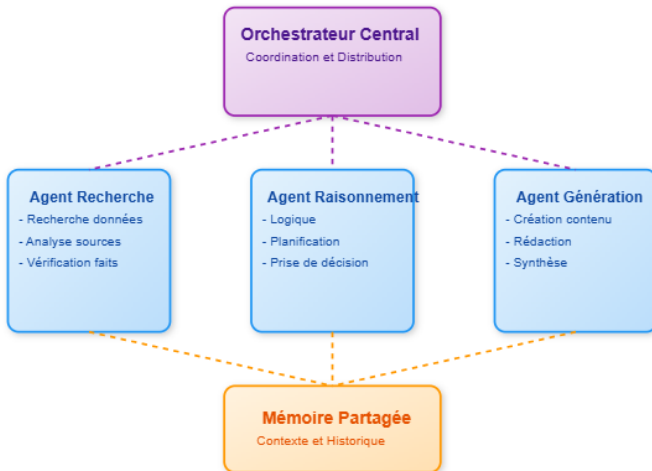
Principe :

- Le modèle observe une entrée utilisateur et génère une pensée explicite (**reasoning**).
- Il sélectionne une action (**acting**) basée sur cette pensée et l'exécute.
- Une observation est obtenue en réponse à l'action, servant de contexte pour la prochaine itération.
- Ce cycle de raisonnement et d'action continue jusqu'à atteindre une réponse satisfaisante.

Avantages :

- Réduction des hallucinations en intégrant des informations en temps réel.
- Amélioration de l'interprétabilité grâce aux traces de raisonnement.
- Meilleure capacité d'adaptation dans des environnements dynamiques.

Systèmes multiagents LLMs



Model Context Protocol (MCP)

Model Context Protocol (MCP) est un protocole client/serveur qui formalise l'accès d'un LLM à des capacités externes.

Rôles

- **LLM** : raisonnement, sélection d'actions
- **Client MCP** : orchestration, validation, politiques
- **Serveur MCP** : exposition et exécution des capacités

Principe fondamental :

le LLM ne fait que choisir dans un espace d'actions explicitement déclaré

MCP ne modifie pas la capacité du LLM à raisonner ; il modifie la **structure** de ce qu'il est autorisé à faire.

Modèle formel : espace d'actions contraint

Soit un ensemble d'actions accessibles au LLM :

$$\mathcal{A} = \{a_1, \dots, a_n\}$$

Chaque action a_i (tool) est définie par un contrat :

$$a_i = (\text{name}_i, \Sigma_i^{\text{in}}, \Sigma_i^{\text{out}}, \text{policy}_i)$$

- Σ^{in} : schéma d'entrée (types, contraintes, champs requis)
- Σ^{out} : schéma de sortie (résultat, erreurs)
- policy : permissions, scopes, budgets

Le LLM doit produire un couple valide (a_i, x) avec $x \in \Sigma_i^{\text{in}}$. Toute sortie hors contrat est non exécutable.

Briques MCP : Resources, Tools, Prompts

Un serveur MCP expose trois catégories distinctes :

Resources

- Contexte adressable (URI)
- Lecture contrôlée : fichiers, pages web, tables, logs

Tools

- Fonctions exécutables
- Interfaces strictement typées (JSON Schema)
- Effets réels : I/O, APIs, automatisation

Prompts

- Patrons de raisonnement paramétrés
- Standardisation du comportement entre agents

MCP expose des contrats plutôt que des implémentations.

Cycle d'exécution MCP (raisonnement → action)

Séquence minimale :

1. Discovery

- `list_tools`, `list_resources`, `list_prompts`

2. Sélection

- Le LLM choisit un tool et fournit des arguments conformes au schéma

3. Validation

- Le client valide types, contraintes, policies

4. Exécution

- Le serveur exécute et retourne un résultat structuré

La robustesse du système repose sur la validité des schémas, pas sur l'interprétation du texte.

Ce que MCP apporte réellement

Apports

- Explicitation de l'espace d'actions
- Séparation décision / exécution
- Gouvernance par construction (permissions, budgets)
- Auditabilité via les appels de tools

Non-apports

- Pas plus d'intelligence
- Pas plus d'autonomie
- Pas d'exécution dans le modèle

MCP ne rend pas les LLM plus puissants ; il rend les systèmes agentiques plus maîtrisables.

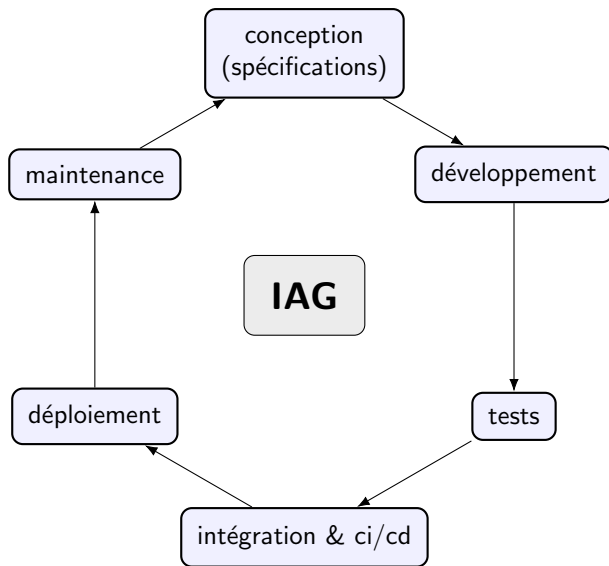
IAG dans le cycle de développement

L'IAG dans le cycle de vie logiciel

L'intelligence artificielle générative ne se limite pas à l'autocomplétion de code : elle peut s'intégrer à chaque étape du cycle de vie logiciel pour assister les équipes.

- **Conception** : clarification des besoins, spécifications, ébauches d'architecture.
- **Développement** : génération et refactor de code, assistance aux patterns.
- **Tests** : génération de tests unitaires et fonctionnels, détection de cas limites.
- **Intégration & CI/CD** : review automatique de PR, génération de changelog et documentation.
- **Déploiement** : notes de version, assistance à l'infrastructure-as-code.
- **Maintenance** : classification des tickets, suggestions de correctifs, documentation à partir du code.

IAG dans le cycle de vie logiciel



Conception et spécifications

Objectif : réduire l'ambiguïté et accélérer le passage du besoin métier à la solution technique.

- Transformer une *user story* en endpoints d'API (routes, verbes, schémas JSON).
- Générer une première ébauche d'architecture (services, flux, dépendances).
- Produire un glossaire ou des critères d'acceptation testables.

Exemple de prompt : « À partir de cette *user story*, propose les endpoints REST, les codes d'erreur et un schéma JSON pour POST /orders. »

Objectif : accélérer l'implémentation tout en améliorant la lisibilité et la maintenabilité.

- Génération de fonctions à partir d'une description en langage naturel.
- Refactor : extraction de sous-fonctions, renommage de variables, simplification de structures.
- Suggestions de design patterns adaptés au contexte.

Exemple : Avant : fonction monolithique de 80 lignes. Après : découpage en `validate()`, `transform()`, `persist()` avec docstrings et exceptions typées.

Objectif : améliorer la couverture et détecter les cas limites dès la phase de développement.

- Génération de tests unitaires directement à partir du code.
- Proposition de cas extrêmes : valeurs limites, erreurs réseau, formats inattendus.
- Création de scénarios fonctionnels dérivés des user stories.

Exemple de prompt : « *Voici la fonction `price(order)`. Génère 8 tests unitaires `pytest` couvrant remise, TVA, devise inconnue, quantité négative, seuils, valeur nulle.* »

Intégration et CI/CD

Objectif : enrichir les pipelines d'intégration continue avec des capacités d'analyse et de génération.

- Review automatique de pull requests : détection de failles de sécurité, de duplications, de style incohérent.
- Génération automatique de tests complémentaires pour valider les commits.
- Production de documentation et de changelogs à partir des messages de commits.

Exemple : Un workflow GitHub Actions envoie le *diff* à un LLM, qui commente directement la PR avec ses suggestions.

Objectif : automatiser et simplifier les tâches de mise en production.

- Assistance à la rédaction de scripts *infrastructure-as-code* (Terraform, Ansible).
- Génération automatique des notes de version lisibles à partir des commits.
- Support aux équipes DevOps pour documenter et communiquer les changements.

Exemple : À partir de commits techniques, l'IAG rédige un changelog clair pour les utilisateurs finaux.

Objectif : accélérer la résolution de problèmes et réduire la dette technique.

- Classification automatique des tickets d'incidents par priorité et catégorie.
- Proposition de correctifs initiaux pour certains bugs.
- Génération de documentation à partir de code existant ou de logs.

Exemple : Un log d'erreur est soumis à l'IAG, qui propose un diagnostic et une piste de correction.

Limites et précautions

Points de vigilance lors de l'intégration de l'IAG :

- **Qualité variable** : sorties parfois incomplètes ou erronées, besoin de supervision humaine.
- **Sécurité et confidentialité** : attention à l'envoi de code ou de données sensibles.
- **Coût et performance** : appels API facturés au token, temps de latence selon les modèles.
- **Dépendance technologique** : choix entre solutions cloud, open source locales ou hybrides.

Message clé : l'IAG est un assistant puissant, mais elle ne remplace pas le rôle critique du développeur et de l'architecte.

Conclusion

Synthèse :

- L'IAG peut intervenir à chaque étape du cycle de vie logiciel.
- Elle augmente la productivité, améliore la documentation et renforce les tests.
- Elle exige des précautions (qualité, sécurité, gouvernance).

Ouverture :

- Quelle phase du cycle de vie tirerait le plus de bénéfice de l'IAG dans vos projets ?
- Quels freins organisationnels ou techniques voyez-vous à son adoption ?

Fine-tuning des LLM

Introduction au fine-tuning des LLMs

Pourquoi fine-tuner un LLM ?

- Adapter un modèle générique (GPT, LLaMA, Mistral) à un domaine spécifique (médical, juridique, entreprise).
- Réduire la dépendance au prompt engineering.
- Améliorer la pertinence, la cohérence et le style des réponses.

Types d'adaptation :

- **Fine-tuning complet** : mise à jour de tous les poids.
- **Instruction tuning** : alignement sur des données question-réponse.
- **PEFT (LoRA, Prefix Tuning...)** : entraînement partiel et efficace.

Fine-tuning complet

Principe : réentraîner l'ensemble des paramètres du modèle.

- **Avantages** : adaptation maximale au domaine, contrôle complet.
- **Inconvénients** :
 - Nécessite une infrastructure GPU massive.
 - Très coûteux en temps et en énergie.
 - Rarement accessible aux entreprises classiques.

Message clé : réservé aux acteurs disposant de moyens importants (Big Tech, laboratoires).

Approches efficaces : LoRA et PEFT

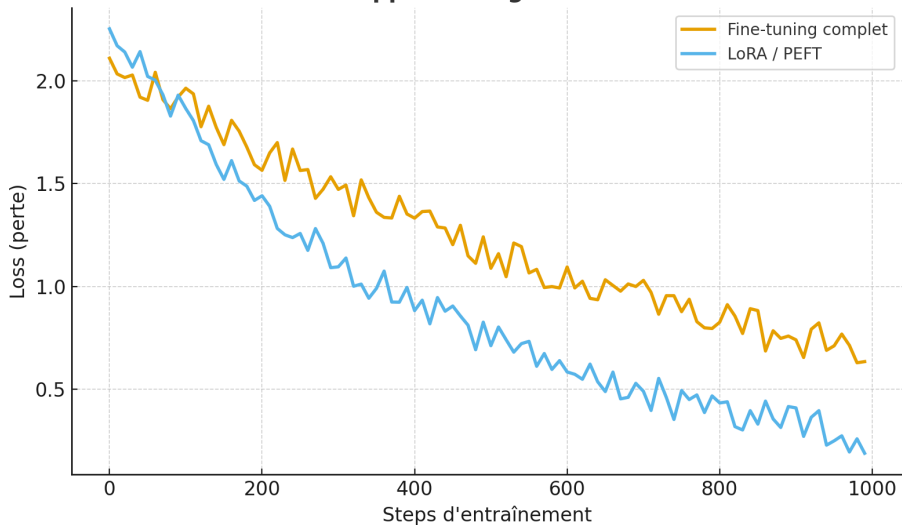
Low-Rank Adaptation (LoRA) : on n'entraîne que de petites matrices « d'adaptation ».

- **PEFT** (Parameter-Efficient Fine-Tuning) regroupe LoRA, Prefix Tuning, Adapters.
- Les poids d'origine sont gelés ; seuls les modules légers sont optimisés.
- **Avantages** :
 - Beaucoup moins coûteux.
 - Stockage des « adapters » léger (quelques Mo).
 - Facile à combiner avec différents modèles de base.



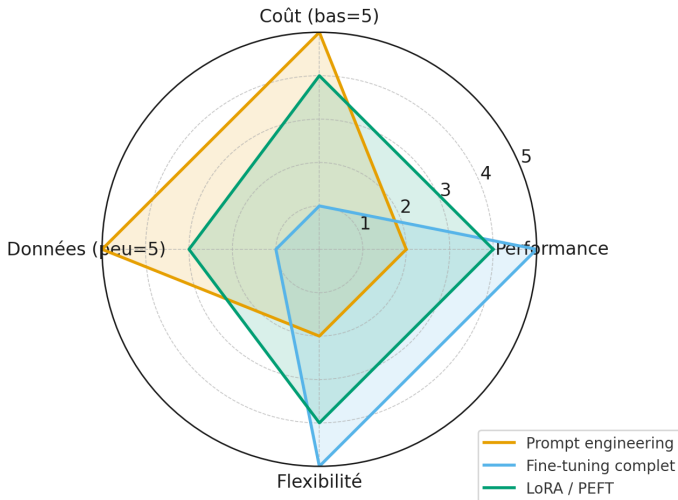
Full fine tuning vs LoRA

Courbes d'apprentissage : full FT vs LoRA



Comparatif des approches

Comparatif d'approches pour adapter un LLM



Outils utilisés dans la pratique

Écosystème Hugging Face et PyTorch :

- **PyTorch** : framework de référence pour l'entraînement des LLMs.
- **Transformers** : chargement et utilisation des modèles pré-entraînés.
- **PEFT** : implémentation de LoRA et autres techniques paramètre-efficaces.
- **Trainer** : API haut-niveau pour gérer l'entraînement.
- **Accelerate** : distribution sur GPU/TPU, multi-device, mixed precision.

Ces briques se combinent pour simplifier et accélérer le fine-tuning.

Le rôle de Trainer

Trainer est une abstraction pour simplifier l'entraînement.

- Gère la boucle d'entraînement : epochs, batchs, évaluation, checkpoints.
- Supporte nativement les modèles Transformers et PEFT.
- Intègre le logging (TensorBoard, Weights & Biases).

Exemple :

- Fournir le modèle LoRA + datasets.
- Définir `TrainingArguments`.
- Appeler `trainer.train()`.

Le rôle d'Accelerate

Accelerate optimise et distribue l'entraînement.

- Masque la complexité du multi-GPU et du multi-node.
- Permet l'entraînement en précision réduite (FP16, bfloat16).
- S'intègre au Trainer ou à une boucle PyTorch personnalisée.

Message clé : tirer le maximum de l'infrastructure matérielle sans changer le code métier.

Pipeline typique avec LoRA + Trainer + Accelerate

Étapes principales :

- ➊ Charger un modèle pré-entraîné (Transformers).
- ➋ Appliquer LoRA (PEFT).
- ➌ Préparer dataset (instructions, Q/A).
- ➍ Définir les arguments d'entraînement (batch size, epochs).
- ➎ Lancer avec Trainer, optimisé par Accelerate.
- ➏ Sauvegarder uniquement les adapters LoRA.

Résultat : un modèle spécialisé, léger et facile à déployer.

Quand utiliser LoRA ?

Cas pertinents :

- Domaines spécialisés (finance, médical, juridique).
- Données propriétaires ou confidentielles.
- Adaptation de style (ton, vocabulaire interne).

Limites :

- Dépendance à la qualité du dataset.
- Maintenance des adapters si le modèle de base évolue.

Conclusion

Résumé :

- Le fine-tuning complet est rare et coûteux.
- LoRA/PEFT rend le fine-tuning accessible aux entreprises.
- Trainer et Accelerate facilitent et optimisent le processus.

Message clé : avec LoRA + Trainer + Accelerate, il est possible d'adapter un LLM aux besoins métiers sans infrastructure démesurée.

Enjeux éthiques, sécurité et conformité de l'IA

Pourquoi parler d'éthique et de sécurité ?

L'intelligence artificielle générative ne pose pas seulement des défis techniques : elle soulève des enjeux critiques pour l'entreprise.

- **Sécurité** : risques de fuite de secrets ou d'attaques via les prompts.
- **Confidentialité** : conformité au RGPD, respect des données sensibles.
- **Propriété intellectuelle** : **propriété des données d'entraînement et du contenu généré par l'IA.**
- **Éthique** : biais, équité, responsabilité dans l'usage.

Message clé : l'adoption de l'IAG doit être accompagnée d'une gouvernance adaptée.

Exemples de menaces :

- **Fuite de secrets** : code ou identifiants envoyés à une API externe.
- **Prompt injection** : manipulation du modèle par un utilisateur malveillant.
- **Dépendance externe** : indisponibilité ou vulnérabilité du fournisseur de LLM.

Bonne pratique : utiliser des proxys, filtrer les prompts et éviter d'envoyer des données sensibles.

Enjeux de confidentialité

Problématique centrale : l'IAG traite souvent des données sensibles ou personnelles.

- **RGPD** : droit à l'oubli, consentement, minimisation des données.
- **Localisation des données** : datacenters UE vs hors UE.
- **Solutions** :
 - anonymisation avant envoi,
 - hébergement de modèles open source en interne,
 - passage par un proxy d'entreprise.

Un enjeu majeur souvent sous-estimé dans l'usage de l'IAG.

- **Origine des données d'entraînement** : risque que le modèle régénère des contenus protégés (texte, code, images).
- **Droits sur les contenus générés** :
 - Dans l'UE, pas de droit d'auteur automatique sur une production 100% IA.
 - L'entreprise doit définir une politique claire (contrats, licences internes).
- **Licences et open source** : vérifier la licence des modèles (Apache 2.0, MIT, restrictions) et des datasets utilisés.
- **Bonnes pratiques** :
 - tracer les sources de données et les outputs,
 - sensibiliser les équipes aux risques de réutilisation non conforme,
 - éviter d'intégrer sans vérification un contenu généré dans des livrables contractuels.

Biais et équité

Constat : les modèles reflètent les biais de leurs données d'entraînement.

- Risque de stéréotypes et de discriminations dans les réponses.
- Impact fort dans les domaines sensibles (recrutement, médical, juridique).
- Besoin d'auditer les modèles dans leur contexte métier.

Message clé : l'évaluation humaine reste indispensable pour corriger les biais.

Questions clés :

- Qui est responsable si une IA génère du code erroné ou trompeur ?
- Comment tracer et auditer les décisions de l'IA ?

Bonnes pratiques :

- Supervision humaine obligatoire pour les tâches critiques.
- Mise en place d'une charte d'usage de l'IAG.
- Définition de rôles et responsabilités clairs.

Durabilité et coûts cachés

Impact énergétique :

- Fine-tuning complet = forte consommation GPU et empreinte carbone.
- LoRA/PEFT = alternatives plus sobres, adaptées aux entreprises.

Coûts cachés :

- Facturation à l'usage (tokens, appels API).
- Dépendance à un fournisseur unique.

Message clé : arbitrer entre innovation, budget et responsabilité environnementale.

Conclusion et bonnes pratiques

Synthèse :

- Les enjeux de sécurité, confidentialité et éthique sont incontournables.
- La gouvernance doit accompagner toute intégration de l'IAG.
- Commencer petit, superviser, documenter et auditer.

Bonne pratique : intégrer les aspects éthiques dès la conception des projets IAG.

Merci de votre attention

redha.moulla@axia-conseil.com